1) Цель

   Укрепить навыки обработки данных

   Познакомиться с библиотекой sklearn

   Изучить работу модели линейной регрессии

   Изучить работу оптимизаторов на основе градиентного спуска

2) Задание

   Практические задания приведены в Jupyter-ноутбуке exercises.ipynb.

   Рекомендуется выполнять задания по очереди.

3) Результат выполнения

   В результате были выполнены все обязательные задания и два бонусных заданий на 3 балла. Задание прошло автоматическое тестирование на 11.0 / 8.0

4) Исходный код

   exercises.ipynb
   # # Часть 0. Загрузка и предобработка данных

   # ## Шаг 1. Импорты

   # In[1]:

   import numpy as np
   import pandas as pd
   import matplotlib.pyplot as plt
   import seaborn as sns

   import sklearn

   # ## Шаг 2. Скачивание датасета

   # Перейдите по ссылке из личного кабинета на Google Таблицу со списком студентов. Найдите своё ФИО в списке и запомните соответствующий порядковый номер. Заполните его в ячейке ниже и выполните ячейку. Если вы не можете найти себя в списке, обратитесь к своему преподавателю.

   # In[2]:

```
### BEGIN YOUR CODE

Student_ID = 7

### END YOUR CODE


# Теперь выполните следующую ячейку. Она вычислит номер задания и выведет его.

# In[3]:


datasets = [
    (
        'Combined Cycle Power Plant',
        'https://archive.ics.uci.edu/dataset/294/combined+cycle+power+plant',
        'PE',
        85, 15
    ),
    (
        'Beijing PM2.5',
        'https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data',
        'pm2.5',
        80, 20
    ),
    (
        'Airfoil Self-Noise',
        'https://archive.ics.uci.edu/dataset/291/airfoil+self+noise',
        'scaled-sound-pressure',
        90, 10
    ),
    (
        'Energy Efficiency',
        'https://archive.ics.uci.edu/dataset/242/energy+efficiency',
        'Y1, Y2 (здесь две целевые переменные)',
        90, 10
    ),
    (
        'Bike Sharing',
```

```
        'https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset',
        'cnt',
        90, 10
    ),
    (
        'Forest Fires',
        'https://archive.ics.uci.edu/dataset/162/forest+fires',
        'area',
        90, 10
    ),
    (
        'Concrete Compressive Strength',
        'https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength',
        'Concrete compressive strength',
        90, 10
    )
]

dataset_id = None if Student_ID is None else (Student_ID - 1) % len(datasets)
if dataset_id is None:
    print("ОШИБКА! Не указан порядковый номер студента в списке группы.")
else:
    print(f"Информация о датасете '{datasets[dataset_id][0]}' доступна по
    следующей ссылке: {datasets[dataset_id][1]}")
    print(f"Целевая переменная: {datasets[dataset_id][2]}")
    print(f"Разделение выборки на train+val/test:
    {datasets[dataset_id][3]}%/{datasets[dataset_id][4]}%")



# ## Шаг 3. Загрузка и обработка датасета

# Скачайте датасет по ссылке выше и считайте его в датафрейм `df`.
#
# Большинство датасетов можно загрузить через `pd.read_csv(...)`. Если файл
архивирован — распакуйте его вручную или воспользуйтесь `zipfile` / `requests`.

# In[4]:



### BEGIN YOUR CODE
```

df = pd.read_csv("Concrete_Data.csv")

df.info()
END YOUR CODE

Обработайте датасет. В обработке вы имеете почти полную свободу — важно побить бейзлайн.
#
Что НЕОБХОДИМО сделать (иначе тесты не пройдут):
- Обработать пропуски (удалить строки / заполнить средним / медианой — на ваш выбор)
- Закодировать категориальные признаки (например, `pd.get_dummies` или `LabelEncoder`)
#
Что ПОЛЕЗНО сделать (помогает побить бейзлайн):
- Удалить дубликаты
- Обработать выбросы
- Удалить признаки с низкой информативностью (посмотрите на корреляции)
- Добавить новые признаки (feature engineering)
- Стандартизировать числовые признаки
#
> **Подсказка:** [sklearn.preprocessing](https://scikit-learn.org/stable/api/sklearn.preprocessing.html) содержит `StandardScaler`, `MinMaxScaler`, `LabelEncoder`, `OneHotEncoder` и другие полезные инструменты.
#
> **Важно про стандартизацию целевой переменной:** если вы стандартизируете `y`, перед сохранением результатов в `y_pred_*` нужно вернуть предсказания к исходному масштабу. Иначе MAPE будет считаться неверно и тесты провалятся. Сохраните параметры стандартизации (`mean`, `std`) для обратного преобразования.
#
ВАЖНО: обработанный по обязательным пунктам датафрейм запишите в `df_base`, дополнительно улучшенный — в `df_processed`. Не перезаписывайте эти переменные и `df`.

In[5]:

BEGIN YOUR CODE

```
df_base = df.copy()

### END YOUR CODE


# In[6]:


### BEGIN YOUR CODE

from sklearn.preprocessing import StandardScaler

_df = df.copy()

# Удаляем дубликаты
_df = _df.drop_duplicates().reset_index(drop=True)

# Убираем целевой признак из общего списка
target_col = _df.columns[-1]
selected_cols = [c for c in _df.columns if c != target_col]

# Корреляция Спирмена между признаками и целевым признаком
spearman_corr = (
    _df.corr(method="spearman")[target_col]
    .drop(labels=target_col)
    .sort_values(key=lambda s: s.abs(), ascending=False)
)
print("Корреляция по Спирмену признаков к целевому:")
print(spearman_corr)

# Удаляем признаки с корреляцией < 0.1
low_corr_cols = spearman_corr[spearman_corr.abs() < 0.1].index.tolist()
_df = _df.drop(columns=low_corr_cols, errors="ignore")

print("\nУдалены признаки:", *low_corr_cols)

### END YOUR CODE


# In[7]:
```

```
selected_cols = [c for c in _df.columns if c != target_col]

spearman_corr = _df[selected_cols].corr(method="spearman")

fig_heatmap_spearman, ax = plt.subplots()

sns.heatmap(
    spearman_corr,
    annot=True,
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    center=0,
    ax=ax
)

# Новый признак из воды и пластификатора
ratio_col = "water_superplasticizer_ratio"
water_col = "Water  (component 4)(kg in a m^3 mixture)"
plastic_col = "Superplasticizer (component 5)(kg in a m^3 mixture)"
ratio_values = _df[water_col] / _df[plastic_col]

insert_pos = _df.columns.get_loc(target_col)
_df.insert(insert_pos, ratio_col, ratio_values)

_df[ratio_col] = _df[ratio_col].replace([np.inf, -np.inf], 0)

selected_cols = [c for c in _df.columns if c != target_col]

# Стандартизация
scaler = StandardScaler()
_df[selected_cols] = scaler.fit_transform(_df[selected_cols])

# Удаление выбросов по правилу трёх сигм
df_mask = (_df[selected_cols].abs() > 3).any(axis=1)
_df = _df[~df_mask].reset_index(drop=True)

df_processed = _df.copy()

df_processed
```

Часть 1. Линейная регрессия (scikit-learn)

Что такое линейная регрессия?
#
Линейная регрессия — это модель, которая предсказывает числовое значение `y` как **линейную комбинацию признаков** `x`:
#
$$y = \mathbf{x} \boldsymbol{\theta} + \theta_0$$
#
где $\boldsymbol{\theta}$ — вектор весов (коэффициентов), θ_0 — свободный член (смещение, bias/intercept).
#
Как модель обучается? Она подбирает $\boldsymbol{\theta}$ и θ_0 так, чтобы минимизировать **полную функцию потерь** (функционал качества, эмпирический риск). Чаще всего используют **MSE (Mean Squared Error)**:
#
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$
#
Чем меньше MSE — тем ближе предсказания к реальным значениям.
#
Как оценивать качество? В этой работе используется **MAPE (Mean Absolute Percentage Error)**:
#
$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$
#
MAPE удобна тем, что выражается в процентах: MAPE = 0.05 означает среднюю ошибку 5%. Она позволяет сравнивать качество моделей на разных датасетах с разными масштабами целевой переменной.

Шаг 1. Разделение на train / val / test

Зачем три выборки?
#
| Выборка | Назначение |
|---|---|
| **Train** | Обучение модели (подбор весов) |

| **Val (validation)** | Подбор гиперпараметров (например, скорость обучения, количество эпох) без «подглядывания» в тест |
| **Test** | Финальная оценка качества — трогается **только один раз** в самом конце |
#
Если настраивать гиперпараметры сразу по test, то модель фактически «видит» тестовые данные и реальная ошибка на новых данных окажется хуже.
#
> **Размер test-выборки** указан в вашем варианте задания (выше, рядом со ссылкой на датасет). Используйте `train_test_split` из sklearn.

In[8]:

```
### BEGIN YOUR CODE

# Разделяем на признаки и целевую переменную
X_base = df_base.drop(columns=target_col)
y_base = df_base[target_col]

X_processed = df_processed[selected_cols]
y_processed = df_processed[target_col]

### END YOUR CODE
```

In[9]:

```
from sklearn.model_selection import train_test_split

### BEGIN YOUR CODE

# Для base: только train и test (val не нужен, гиперпараметров нет)
X_train_base, X_test_base, y_train_base, y_test_base = train_test_split(
    X_base,
    y_base,
    test_size=0.1,
    random_state=42,
)
```

```
### END YOUR CODE


# In[10]:


### BEGIN YOUR CODE

# Для processed: сначала отделить test, затем из оставшегося выделить val
X_train_processed, X_test_processed, y_train_processed, y_test_processed =
train_test_split(
    X_processed,
    y_processed,
    test_size=0.10,
    random_state=42,
)

X_train_processed, X_val_processed, y_train_processed, y_val_processed =
train_test_split(
    X_train_processed,
    y_train_processed,
    test_size=0.20,
    random_state=42,
)

### END YOUR CODE


# In[11]:


# Промежуточная проверка — убедитесь, что размеры выглядят разумно
print("Base:")
print(f"  X_train_base:    {X_train_base.shape}")
print(f"  X_test_base:     {X_test_base.shape}")
print()
print("Processed:")
print(f"  X_train_processed: {X_train_processed.shape}")
print(f"  X_val_processed:   {X_val_processed.shape}")
print(f"  X_test_processed:  {X_test_processed.shape}")
```

Шаг 2. Обучение линейной регрессии

Ниже приведён **бейзлайн** — простейшая линейная регрессия без какой-либо дополнительной обработки данных. Ваша задача — **побить его**, то есть получить `MAPE(y_pred_processed) < MAPE(y_pred_base)`.
#
Для этого используйте `df_processed` и экспериментируйте с:
- предобработкой данных (стандартизация, удаление выбросов, feature engineering)
- подбором гиперпараметров на `val`-выборке
#
> **Подсказка:** гиперпараметры подбираются на val, финальная оценка считается на test. Не оптимизируйте напрямую по test!

In[12]:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_percentage_error
```

In[13]:

```
def baseline():
    """Бейзлайн: LinearRegression без предобработки, обученный на
    base-данных."""
    lin_reg = LinearRegression()
    lin_reg.fit(X_train_base, y_train_base)
    return lin_reg.predict(X_test_base)
```

In[14]:

```
y_pred_base = baseline()
print(f"Baseline MAPE: {mean_absolute_percentage_error(y_test_base,
y_pred_base):.4f}")
```

Обучите улучшенную модель и запишите предсказания на тестовой выборке в `y_pred_processed`.

In[15]:

```
### BEGIN YOUR CODE
from sklearn.linear_model import Ridge

alphas = [1e-2, 1e-1, 1, 10, 100]
best_alpha = None
best_val_mape = np.inf

for alpha in alphas:
    model = Ridge(alpha=alpha)
    model.fit(X_train_processed, y_train_processed)
    y_val_pred = model.predict(X_val_processed)

    val_mape = mean_absolute_percentage_error(y_val_processed, y_val_pred)
    if val_mape < best_val_mape:
        best_val_mape = val_mape
        best_alpha = alpha

X_trainval_processed = pd.concat([X_train_processed, X_val_processed], axis=0)
y_trainval_processed = pd.concat([y_train_processed, y_val_processed], axis=0)

best_model = Ridge(alpha=best_alpha)
best_model.fit(X_trainval_processed, y_trainval_processed)

y_pred_processed = best_model.predict(X_test_processed)

print(f"Best alpha: {best_alpha}")

### END YOUR CODE

print(f"Processed MAPE: {mean_absolute_percentage_error(y_test_processed,
y_pred_processed):.4f}")
print(f"Baseline MAPE:  {mean_absolute_percentage_error(y_test_base,
y_pred_base):.4f}")
```

Часть 2. Градиентный спуск (numpy)

Что такое градиентный спуск?
#
Представьте, что полная функция потерь — это холмистый ландшафт, а мы стоим на нём и хотим спуститься в самую низкую точку (минимум). Градиент указывает направление наибольшего **роста** функции, поэтому мы идём в противоположном направлении:
#
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \nabla_{\boldsymbol{\theta}} Q$$
#
где η — **learning rate** (размер шага), $\nabla_{\boldsymbol{\theta}} Q$ — градиент полной функции потерь по весам.
#
Стохастический мини-батчевый градиентный спуск (SGD) отличается тем, что градиент вычисляется не на всей выборке, а на случайном **мини-батче** — случайной подвыборке небольшого размера. Это быстрее и часто сходится лучше.
#

#
Градиенты MSE для линейной модели
#
Формулы градиентов $\dfrac{\partial Q}{\partial \boldsymbol{\theta}}$ и $\dfrac{\partial Q}{\partial \theta_0}$ для модели $\hat{y} = X\boldsymbol{\theta} + \theta_0$ и полной функции потерь $Q = \frac{1}{n}\|\hat{y} - y\|^2$ выводятся в теоретической части (вопрос Q1).
#
Именно их нужно реализовать в методе `step()` класса `CustomSGD`.
#
> **Интерфейс:** `CustomSGD.__init__(params, lr)` принимает список параметров модели `[W, b]` и хранит их как атрибут, где `W` — "веса" модели, т.е. параметры $\boldsymbol{\theta}$; а `b` — смещение θ_0. Метод `step(x_batch, y_batch)` вычисляет градиенты и обновляет параметры **in-place** (через `params[0] -= ...`).

Шаг 1. Реализация CustomSGD

In[16]:

```
### BEGIN YOUR CODE

class CustomSGD:
    def __init__(self, params, lr):
        """
        params: список параметров модели, например [W, b]
                Изменения в params автоматически отражаются в модели,
                поэтому достаточно хранить ссылку на этот список.
        lr: learning rate (скорость обучения), положительное число
        """
        self.params = params
        self.lr = lr

    def step(self, x_batch, y_batch):
        """
        Один шаг SGD: вычислить градиенты MSE на батче и обновить params
        in-place.

        x_batch: numpy array размера (batch_size, n_features)
        y_batch: numpy array размера (batch_size,)
        """
        y_batch = y_batch.reshape(-1, 1)
        y_pred = x_batch @ self.params[0] + self.params[1]
        y_err = y_pred - y_batch



        n = len(y_batch)
        grad_W = ((2/n) * x_batch.T @ y_err)
        grad_b = (2/n) * y_err.sum()

        self.params[0] -= self.lr * grad_W
        self.params[1] -= self.lr * grad_b

### END YOUR CODE



# # Часть 3. Линейная регрессия (numpy)

# ## Шаг 1. Реализация CustomLinearRegression
```

Реализуйте класс линейной регрессии на numpy. Класс должен поддерживать два режима обучения, задаваемых параметром `solver`:
#
- `solver='SGD'` — обучение через `CustomSGD` (стохастический градиентный спуск)
- `solver='MNK'` — обучение методом наименьших квадратов (аналитическое решение)
#
Метод наименьших квадратов (МНК) даёт точное решение задачи минимизации MSE без итераций:
#
$$\boldsymbol{\theta} = (X^\top X)^{-1} X^\top y$$
#
Для его реализации удобно расширить матрицу признаков столбцом единиц (`np.hstack([X, np.ones(...)])`), тогда последний элемент вектора весов будет соответствовать свободному члену b.
#
При реализации через numpy используйте `np.linalg.solve(A, b)` (вместо явного обращения матрицы через `np.linalg.inv`) — это устойчивее численно.
#
> **Бонусный параметр** (для бонусного задания *): `regularization` задаёт тип регуляризации (`'ridge'` или `'lasso'`), `tau` — её коэффициент.

In[17]:

```
np.random.seed(42)

class CustomLinearRegression:
### BEGIN YOUR CODE
    """
    Линейная регрессия y = Xw + b, реализованная на numpy.

    Параметры:
        solver       : 'SGD' (градиентный спуск) или 'MNK' (метод наименьших
        квадратов)
        regularization : None, 'ridge' (L2) или 'lasso' (L1) — только для solver='SGD'
        tau          : коэффициент регуляризации (используется при regularization
        != None)
    """
    def __init__(self, solver='SGD', regularization=None, tau=0.01):
```

```
        self.solver = solver
        self.regularization = regularization
        self.tau = tau

    def fit(self, X, y):
        """
        Обучить модель.
        X: numpy array (n_samples, n_features)
        y: numpy array (n_samples,) или (n_samples, n_targets)
        """
        X = np.asarray(X)
        y = np.asarray(y)
        if self.solver == 'SGD':
            n_epochs = 100
            batch_size = 32
            learning_rate = 0.01

            n_samples = X.shape[0]

            W = np.zeros((X.shape[1], 1))
            b = 0.0
            self.params = [W, b]
            optimizer = CustomSGD(self.params, lr=learning_rate)

            for _ in range(n_epochs):
                order = np.random.permutation(n_samples)
                for start in range(0, n_samples, batch_size):
                    idx = order[start : start + batch_size]
                    old_param = self.params[0].copy()
                    optimizer.step(X[idx], y[idx])
                    if self.regularization == 'ridge':
                        self.params[0] -= learning_rate * 2 * self.tau * old_param
                    elif self.regularization == 'lasso':
                        self.params[0] -= learning_rate * self.tau * np.sign(old_param)

        elif self.solver == 'Adam':
            n_epochs = 1000
            batch_size = 32
            learning_rate = 0.01

            n_samples = X.shape[0]
```

```
            W = np.zeros((X.shape[1], 1))
            b = 0.0
            self.params = [W, b]
            optimizer = CustomAdam(self.params, lr=learning_rate)

            for _ in range(n_epochs):
                order = np.random.permutation(n_samples)
                for start in range(0, n_samples, batch_size):
                    idx = order[start : start + batch_size]
                    old_param = self.params[0].copy()
                    optimizer.step(X[idx], y[idx])

        elif self.solver == 'MNK':
            n_samples = X.shape[0]

            X_aug = np.hstack([X, np.ones((n_samples, 1))])

            A = X_aug.T @ X_aug
            rhs = X_aug.T @ y

            w_aug = np.linalg.solve(A, rhs)

            W = w_aug[:-1]
            b = float(w_aug[-1])

            self.params = [W, b]


    def predict(self, X):
        """
        Вернуть предсказания для X.
        X: numpy array (n_samples, n_features)
        """
        return X @ self.params[0] + self.params[1]

### END YOUR CODE


# ## Шаг 2. Обучение CustomLinearRegression с помощью SGD
```

Обучите модель с помощью `CustomSGD` и запишите предсказания на тестовой выборке в `y_pred_custom`.
#
Что нужно сделать:
1. Инициализировать параметры модели `W` и `b`
2. Реализовать мини-батчевый обучающий цикл (эпохи → батчи → шаг оптимизатора) — он ляжет в основу `fit()` в `CustomLinearRegression`, но здесь дополнительно отслеживается val loss
3. На каждой эпохе сохранять средний MSE на train и val выборках
4. После обучения построить графики функции потерь на train и val выборках
5. Вычислить MAPE на тестовой выборке и сравнить с sklearn-бейзлайном
#
Ниже приведён шаблон обучающего цикла — заполните недостающие части.

In[18]:

Шаблон обучающего цикла — заполните недостающие части

n_epochs = 100
batch_size = 32
learning_rate = 0.01

BEGIN YOUR CODE

X_train = np.asarray(X_train_processed)
y_train = np.asarray(y_train_processed)
X_val = np.asarray(X_val_processed)
y_val = np.asarray(y_val_processed)

n_samples = X_train.shape[0]

W = np.zeros((X_train.shape[1], 1))
b = 0.0
params = [W, b]

optimizer = CustomSGD(params, lr=learning_rate)

train_losses = []
val_losses = []

```
for _ in range(n_epochs):
    order = np.random.permutation(n_samples)
    for start in range(0, n_samples, batch_size):
        idx = order[start : start + batch_size]
        optimizer.step(X_train[idx], y_train[idx])

    pred_tr = X_train @ W + b
    epoch_loss = np.mean((pred_tr - y_train) ** 2)
    train_losses.append(epoch_loss)

    pred_val = X_val @ W + b
    val_losses.append(np.mean((pred_val - y_val) ** 2))

### END YOUR CODE

# График функции потерь
plt.figure(figsize=(8, 4))
plt.plot(train_losses, label='Train loss')
plt.plot(val_losses, label='Val loss')
plt.xlabel('Epoch')
plt.ylabel('MSE')
plt.title('Кривая обучения')
plt.legend()
plt.tight_layout()
plt.show()


# In[19]:


### BEGIN YOUR CODE

model_custom = CustomLinearRegression(solver='SGD')
model_custom.fit(X_train_processed, y_train_processed)

y_pred_custom = model_custom.predict(X_test_processed)

### END YOUR CODE

print(f"Custom SGD MAPE: {mean_absolute_percentage_error(y_test_processed,
y_pred_custom):.4f}")
```

```
print(f"Baseline MAPE:   {mean_absolute_percentage_error(y_test_base,
y_pred_base):.4f}")


# ## Шаг 3. Метод наименьших квадратов (МНК)

# Обучите `CustomLinearRegression` с `solver='MNK'` на вашем датасете.
#
# МНК даёт **точное** решение без итераций — это удобно для небольших
датасетов. На очень больших данных инвертирование матрицы $(X^\top X)$
может быть медленным или численно нестабильным — в таком случае возьмите
случайную долю данных.
#
# Сравните результаты МНК с SGD: они должны быть близкими (теоретически
МНК — глобальный минимум MSE).
#
# Запишите предсказания в `y_pred_mnk`.

# In[20]:


### BEGIN YOUR CODE

model_custom_MNK = CustomLinearRegression(solver='MNK')
model_custom_MNK.fit(X_train_processed , y_train_processed)

y_pred_mnk = model_custom_MNK.predict(X_test_processed)

### END YOUR CODE

print(f"MHK MAPE:      {mean_absolute_percentage_error(y_test_processed,
y_pred_mnk):.4f}")
print(f"Custom SGD MAPE: {mean_absolute_percentage_error(y_test_processed,
y_pred_custom):.4f}")
print(f"Baseline MAPE:   {mean_absolute_percentage_error(y_test_base,
y_pred_base):.4f}")


# In[21]:
```

```
# Проверка реализации МНК
sk_model = LinearRegression().fit(X_train_processed, y_train_processed)
y_pred_sk = sk_model.predict(X_test_processed)

assert np.allclose(y_pred_mnk, y_pred_sk, atol=1e-4), \
    "Проверьте реализацию МНК: предсказания расходятся с эталонной
    реализацией sklearn."
print(" МНК реализован корректно")



# # Бонус *. Регуляризация (Ridge и LASSO) [+10 баллов]

# **Зачем нужна регуляризация?**
#
# Модель может переобучиться: идеально подстроиться под тренировочные данные, но плохо работать на новых. Регуляризация добавляет к функции потерь **штраф за большие веса**, что вынуждает модель оставаться проще.
#
# **Ridge (L2-регуляризация):**
#
# $$Q_{\text{Ridge}} = \text{MSE} + \tau \|\boldsymbol{\theta}\|_2^2 = \text{MSE} + \tau \sum_j \theta_j^2$$
#
# **LASSO (L1-регуляризация):**
#
# $$Q_{\text{LASSO}} = \text{MSE} + \tau \|\boldsymbol{\theta}\|_1 = \text{MSE} + \tau \sum_j |\theta_j|$$
#
# LASSO обнуляет незначимые веса (делает модель разреженной), Ridge лишь уменьшает их.
#
# Дополнительные слагаемые в градиенте выводятся в теоретической части (вопрос Q2). Используйте их при реализации шага SGD с регуляризацией.
#
# **Что нужно сделать:** добавить поддержку регуляризации в `CustomLinearRegression`. Интерфейс:
#
# ```python
# model_ridge = CustomLinearRegression(solver='SGD', regularization='ridge', tau=0.01)
```

model_lasso = CustomLinearRegression(solver='SGD', regularization='lasso', tau=0.01)
```
#
Запишите предсказания в `y_pred_ridge` и `y_pred_lasso`.

In[22]:

BEGIN YOUR CODE

model_ridge = CustomLinearRegression(solver='SGD', regularization='ridge', tau=0.01)
model_ridge.fit(X_train_processed , y_train_processed)
model_lasso = CustomLinearRegression(solver='SGD', regularization='lasso', tau=0.01)
model_lasso.fit(X_train_processed , y_train_processed)

y_pred_ridge = model_ridge.predict(X_test_processed)
y_pred_lasso = model_lasso.predict(X_test_processed)

END YOUR CODE

print(f"Ridge MAPE: {mean_absolute_percentage_error(y_test_processed, y_pred_ridge):.4f}")
print(f"LASSO MAPE: {mean_absolute_percentage_error(y_test_processed, y_pred_lasso):.4f}")
print(f"Baseline MAPE: {mean_absolute_percentage_error(y_test_base, y_pred_base):.4f}")

Бонус **. Оптимизатор Adam [+20 баллов]

Adam (Adaptive Moment Estimation) — улучшение SGD, которое адаптирует learning rate для каждого параметра отдельно. Он комбинирует идеи **Momentum** (накопление градиентов) и **RMSProp** (нормировка по скользящей дисперсии):
#
$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{(первый момент)}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{(второй момент)}$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(коррекция смещения)}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$
#
Типичные значения гиперпараметров: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.
#
Реализуйте `CustomAdam` и обучите `CustomLinearRegression` с его помощью. Запишите предсказания в `y_pred_adam`.

In[23]:

```
class CustomAdam:
    def __init__(self, params, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
        ### BEGIN YOUR CODE
        self.params = params
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps
        self.m = [np.zeros_like(p) for p in self.params]
        self.v = [np.zeros_like(p) for p in self.params]
        self.t = 0
        ### END YOUR CODE

    def step(self, x_batch, y_batch):
        ### BEGIN YOUR CODE
        y_batch = y_batch.reshape(-1, 1)
        y_pred = x_batch @ self.params[0] + self.params[1]
        y_err = y_pred - y_batch

        n = len(y_batch)
        grad_W = (2 / n) * (x_batch.T @ y_err)
        grad_b = (2 / n) * y_err.sum()

        self.t += 1
```

```
        # Параметры весов
        self.m[0] = self.beta1 * self.m[0] + (1 - self.beta1) * grad_W
        self.v[0] = self.beta2 * self.v[0] + (1 - self.beta2) * (grad_W ** 2)

        m_hat_W = self.m[0] / (1 - self.beta1 ** self.t)
        v_hat_W = self.v[0] / (1 - self.beta2 ** self.t)

        self.params[0] -= self.lr * m_hat_W / (np.sqrt(v_hat_W) + self.eps)

        # Параметры смещения
        self.m[1] = self.beta1 * self.m[1] + (1 - self.beta1) * grad_b
        self.v[1] = self.beta2 * self.v[1] + (1 - self.beta2) * (grad_b ** 2)

        m_hat_b = self.m[1] / (1 - self.beta1 ** self.t)
        v_hat_b = self.v[1] / (1 - self.beta2 ** self.t)

        self.params[1] -= self.lr * m_hat_b / (np.sqrt(v_hat_b) + self.eps)
        ### END YOUR CODE


# Обучите `CustomLinearRegression` с помощью `CustomAdam` (аналогично шагу 2 части 3).

# In[24]:


### BEGIN YOUR CODE

model_custom_adam = CustomLinearRegression(solver='Adam')
model_custom_adam.fit(X_train_processed, y_train_processed)
y_pred_adam = model_custom_adam.predict(X_test_processed)

### END YOUR CODE

print(f"Adam MAPE:     {mean_absolute_percentage_error(y_test_processed,
y_pred_adam):.4f}")
print(f"Baseline MAPE: {mean_absolute_percentage_error(y_test_base,
y_pred_base):.4f}")
```

5) Выводы

В процессе выполнения лабораторной работы были получены практические навыки по реализации линейной регрессии с помощью градиентного спуска, метода наименьших квадратов и дополнительных регуляризаций L1 L2.